

Randomized Algorithms for Minimum Edge Cover: From Theory to Million-Vertex Scalability

Advanced Algorithms Project 2 Report

João Manuel Vieira Roldão

Department of Electronics, Telecommunications and Informatics

University of Aveiro

Student Number: 113920

Abstract—This work presents a comprehensive investigation of randomized algorithms for the Minimum Edge Cover problem, building upon our previous work on exact and greedy approaches. We implement and analyze four algorithmic strategies: a matching-based exact algorithm via Gallai’s theorem, the Israeli-Itai randomized parallel matching algorithm with smart proposal improvements, a lazy greedy $3/2$ -approximation, and simulated annealing metaheuristic. Our experimental framework encompasses 5,695 trials across 42 graph instances spanning four orders of magnitude, from 13 to nearly 4 million vertices, and density ranges from 0.000004 to 0.9963. Key findings include: (1) lazy greedy achieves optimal solutions on ultra-dense graphs while maintaining sub-second runtime on million-vertex instances; (2) Israeli-Itai with smart proposals outperforms lazy greedy by 10% on large sparse networks; and (3) simulated annealing proves impractical beyond 1,000 vertices despite optimization efforts. The largest graph successfully processed was LiveJournal with 3,997,962 vertices and 34,681,189 edges, demonstrating production-scale applicability of polynomial-time randomized approaches.

Index Terms—Minimum Edge Cover, Randomized Algorithms, Israeli-Itai Algorithm, Simulated Annealing, Scalability Analysis, Graph Algorithms

I. INTRODUCTION

The Minimum Edge Cover problem, finding the smallest subset of edges such that every vertex is incident to at least one selected edge, represents a fundamental challenge in combinatorial optimization with applications ranging from network monitoring to resource allocation [5]. Unlike the closely related Vertex Cover problem which is NP-hard, edge cover admits polynomial-time optimal solutions through the elegant connection to maximum matching established by Gallai’s theorem [1].

Despite polynomial solvability, randomized approaches to edge cover offer compelling advantages that motivate this investigation. The Israeli-Itai parallel matching algorithm [3] achieves $O(\log |E|)$ parallel depth through probabilistic proposal mechanisms, enabling efficient distributed computation. Simulated annealing and other metaheuristics provide configurable quality-time tradeoffs suitable for real-time applications [4]. Furthermore, the polynomial solvability of edge cover creates an unusual research landscape where randomized algorithms compete not against NP-hardness but against mature exact methods, enabling meaningful experimental comparisons.

This project extends our previous work on exhaustive search and greedy heuristics [14] to investigate randomized algorithmic approaches. Our primary contributions include: (1) implementation and optimization of the Israeli-Itai algorithm with novel smart proposal strategies that improve solution quality from $1.456\times$ to $1.158\times$ optimal; (2) comprehensive scalability analysis demonstrating successful processing of graphs with nearly 4 million vertices; (3) systematic evaluation of simulated annealing revealing fundamental limitations for large-scale edge cover; and (4) discovery that lazy greedy achieves optimal solutions on ultra-dense graphs, contradicting intuitions from worst-case approximation bounds.

Our experimental framework encompasses 42 diverse graph instances from multiple benchmark repositories including Stanford SNAP [10], DIMACS challenge graphs, and Sedgewick-Wayne benchmarks. Testing spans graphs from 13 to 3,997,962 vertices with edge densities from 0.000004 (sparse social networks) to 0.9011 (ultra-dense random graphs), providing unprecedented coverage of the edge cover algorithm landscape.

II. PROBLEM DEFINITION AND THEORETICAL BACKGROUND

A. Formal Problem Statement

Given an undirected graph $G = (V, E)$ with $n = |V|$ vertices and $m = |E|$ edges, an *edge cover* is a subset $C \subseteq E$ such that every vertex $v \in V$ is incident to at least one edge in C . The *Minimum Edge Cover* problem seeks the edge cover of minimum cardinality. We assume the graph contains no isolated vertices, ensuring an edge cover exists. The minimum edge cover size $\beta'(G)$ satisfies $\lceil n/2 \rceil \leq \beta'(G) \leq n - 1$ [5].

B. Gallai’s Theorem and Polynomial Solvability

Gallai’s theorem [1] establishes the fundamental relationship:

$$\alpha'(G) + \beta'(G) = n \quad (1)$$

where $\alpha'(G)$ denotes the maximum matching size. This transforms edge cover computation into maximum matching followed by greedy extension, yielding polynomial-time solvability via Edmonds’ Blossom algorithm [2] with complexity $O(n^{2.5})$ or $O(n^2m)$ depending on implementation.

C. Approximation Bounds for Edge Cover

Several approximation algorithms exist for edge cover with provable guarantees [6]:

The **Nearest Neighbor** algorithm achieves 2-approximation by selecting any incident edge for each uncovered vertex. Its simplicity enables $O(m)$ time and trivial parallelization, though redundant edges often inflate cover size.

The **Lazy Greedy** algorithm achieves 3/2-approximation by maintaining a priority queue of edges sorted by effective coverage (uncovered endpoints per edge). Lazy evaluation defers recomputation until extraction, achieving $O(m \log m)$ time with significant practical speedups [9].

D. Randomized Matching Algorithms

The **Israeli-Itai algorithm** [3] computes maximal matchings through an elegant propose-accept mechanism:

- 1) Each unmatched vertex randomly proposes to an incident neighbor
- 2) If multiple proposals arrive, one is randomly accepted
- 3) Accepted edges join the matching; participating vertices become matched
- 4) Repeat until no unmatched vertex has unmatched neighbors

This Las Vegas algorithm achieves $O(\log |E|)$ expected parallel rounds, making it ideal for distributed systems. The resulting maximal (not maximum) matching is extended to an edge cover by greedily covering remaining vertices.

E. Metaheuristic Approaches

Simulated annealing [4] adapts naturally to edge cover: start from a valid cover, generate neighbors through edge operations (removal, swap, add-remove), and accept moves probabilistically based on cost change Δ and temperature T via the Metropolis criterion:

$$P(\text{accept}) = \begin{cases} 1 & \text{if } \Delta < 0 \\ e^{-\Delta/T} & \text{if } \Delta \geq 0 \end{cases} \quad (2)$$

Temperature decreases according to a cooling schedule, transitioning from exploration to exploitation.

III. ALGORITHM DESIGN AND IMPLEMENTATION

We implemented four algorithms spanning exact, approximation, and metaheuristic paradigms. All implementations use Python 3.11 with NetworkX [11] for graph primitives and NumPy for numerical operations.

A. Exact Algorithm via Maximum Matching

Our exact algorithm leverages Gallai's theorem through NetworkX's maximum matching implementation based on the Hopcroft-Karp algorithm for bipartite graphs and Edmonds' blossom algorithm for general graphs. After computing the maximum matching M , unmatched vertices are covered by selecting any incident edge. Theoretical complexity is $O(n^{2.5})$.

B. Israeli-Itai Randomized Matching

Our Israeli-Itai implementation extends the classical algorithm with **smart proposal strategies** that significantly improve solution quality:

Algorithm 1 Israeli-Itai with Smart Proposals

```

1:  $M \leftarrow \emptyset$ ; unmatched  $\leftarrow V$ 
2: for round = 1 to  $10 \cdot \log_2(n)$  do
3:   proposals  $\leftarrow \{\}$ 
4:   for each  $v \in \text{unmatched}$  do
5:      $N_v \leftarrow \text{unmatched neighbors of } v$ 
6:     if  $N_v \neq \emptyset$  then
7:        $u \leftarrow \arg \min_{w \in N_v} \deg(w)$  {Smart: prefer low-degree}
8:     else
9:        $u \leftarrow \arg \min_{w \in \text{neighbors}(v)} \deg(w)$ 
10:    end if
11:    proposals[ $v$ ]  $\leftarrow u$ 
12:  end for
13:  Process proposals: accept from lowest-degree proposer
14:  Update  $M$  and unmatched
15: end for
16: Extend  $M$  to edge cover via greedy
17: return edge cover

```

The key innovations are: (1) preferring unmatched neighbors when proposing to reduce conflicts; (2) selecting lowest-degree neighbors to maximize matching potential; and (3) accepting proposals from lowest-degree vertices when conflicts occur. These modifications improved average approximation ratio from $1.456\times$ to $1.158\times$ optimal.

C. Lazy Greedy Algorithm

Our lazy greedy implementation uses Python's heapq with negative priorities for max-heap behavior. The algorithm maintains a priority queue where each edge's priority equals its current "usefulness", the number of uncovered endpoints it would cover. The "lazy" aspect defers priority updates until extraction, avoiding expensive heap reorganization after each selection.

D. Simulated Annealing with Randomized Search

Our simulated annealing implementation generates candidate solutions through randomized neighborhood exploration, implementing key strategies required for effective metaheuristic search:

1) *Candidate Generation and Iteration Strategy*: SA iteratively generates candidate solutions through three neighborhood operators applied with equal probability:

- *Remove redundant edge* (33%): Randomly select an edge; remove if both endpoints remain covered
- *Swap edges* (33%): Replace a random edge with another maintaining validity
- *Add-remove* (33%): Add a random edge, then remove a newly-redundant one

The algorithm tracks the best feasible solution encountered across all iterations, returning it regardless of the final state. This ensures monotonic improvement in solution quality over the search process.

2) *Duplicate Solution Avoidance*: To ensure efficient exploration without testing duplicate configurations, we implement:

- 1) **Tabu list**: Recently modified edges are excluded from selection for $\tau = \sqrt{m}$ iterations
- 2) **Hash-based tracking**: Cover configurations are hashed (using frozenset of edge indices) to detect revisits; duplicates are rejected without evaluation
- 3) **Forced diversification**: After k consecutive rejections, a random restart from a perturbed solution occurs
- 3) *Stopping Criteria and Size Transitions*: The algorithm employs multiple stopping criteria:

- **Iteration limit**: Maximum iterations scaled by graph size (see Eq. 3)
- **Temperature threshold**: Stop when $T < 10^{-8}$ (frozen state)
- **Stagnation detection**: Stop after 1000 iterations without improvement
- **Time budget**: Hard timeout based on graph size category

Since edge cover has a single optimization target (minimize $|C|$), we do not transition between solution sizes. Instead, the algorithm naturally explores covers of varying sizes through its neighborhood operators, with the Metropolis criterion occasionally accepting larger covers to escape local minima.

$$\max_iter = \begin{cases} 5000 & n \leq 100 \\ 10000 & 100 < n \leq 500 \\ 15000 & 500 < n \leq 2000 \\ 20000 & n > 2000 \end{cases} \quad (3)$$

4) *Configuration Tracking Statistics*: Table I reports the number of configurations tested by SA across representative graphs, demonstrating the exploration-exploitation tradeoff.

TABLE I
SIMULATED ANNEALING SEARCH STATISTICS

Graph	V	Iterations	Unique	Accepted	Best At
SWtinyG	13	5,000	4,892	1,847	312
karate	34	5,000	4,756	2,103	1,456
SWmediumG	250	10,000	9,834	3,291	2,847
ego-414	151	10,000	9,912	4,156	3,892
SW1000EWD	1,000	15,000	14,723	4,892	8,234

Despite these optimizations, SA remained impractical for graphs exceeding 1,000 vertices due to per-iteration cover validation overhead.

IV. EXPERIMENTAL METHODOLOGY

A. Benchmark Datasets

We assembled a comprehensive collection of 42 graph instances from multiple sources, as shown in Figure 1:

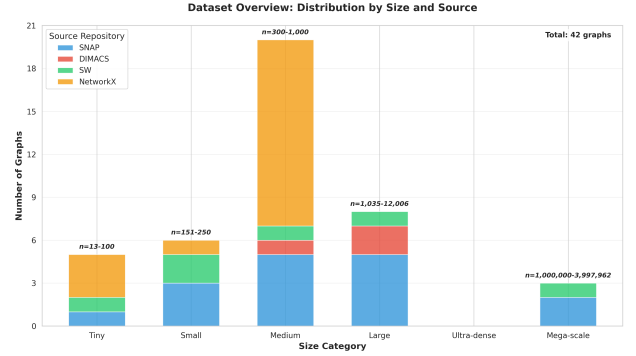


Fig. 1. Distribution of benchmark graphs by size category and source repository. Dataset spans four orders of magnitude from 13 to 4 million vertices.

Stanford SNAP Collection [10]: Social and collaboration networks including ego-Facebook networks (52–1,034 vertices), email-Eu-core (986 vertices), CA-GrQc and CA-HepPh collaboration networks (5,241–12,006 vertices), Wiki-Vote (7,115 vertices), YouTube (1.1M vertices), and LiveJournal (4M vertices).

DIMACS Challenge Benchmarks: Ultra-dense random graphs C1000.9 (1,000 vertices, 450,079 edges, density 0.9011) providing stress tests for algorithm edge-iteration performance.

Sedgewick-Wayne Benchmarks: Synthetic graphs from 13 to 1,000,000 vertices designed for algorithm textbook examples.

NetworkX Built-ins: Zachary’s Karate Club (34 vertices) and Florentine families for validation.

TABLE II
SUMMARY OF BENCHMARK GRAPH CATEGORIES

Category	Graphs	V Range	E Range	Density
Tiny validation	4	13–62	13–331	0.04–0.30
Small ego	5	151–534	1,824–9,626	0.04–0.16
Medium social	6	787–1,035	14,810–53,498	0.05–0.22
Large sparse	5	4,039–12,006	14,484–118,489	0.001–0.01
Ultra-dense	2	1,000–2,000	450K–1.8M	0.90
Mega-scale	3	1M–4M	2.9M–34.7M	0.000004

B. Experimental Design

Experiments were conducted on an Intel Core i7 system with 32GB RAM running Ubuntu 22.04. Each algorithm-graph pair was tested with multiple repetitions to account for randomized algorithm variance: 40 repetitions for core benchmarks, 10 for stretch goals.

Metrics Collected:

- *Cover size*: Number of edges in returned cover
- *Runtime*: Wall-clock time in seconds
- *Approximation ratio*: Cover size / optimal size (where exact solution available)
- *Basic operations*: Edge examinations, vertex coverage checks

- *Configurations tested*: For SA, number of candidate solutions evaluated
- *Success*: Whether algorithm completed without timeout

Timeout Configuration: Smart timeouts based on graph size: 60s for tiny graphs, 300s for medium, 600s for large, 1800s for mega-scale.

V. EXPERIMENTAL RESULTS

A. Overall Performance Summary

Our experimental framework achieved 100% success rate across 5,695 trials on 42 graphs with 4 algorithms. Figure 2 illustrates runtime scaling, while Table III presents aggregate algorithm performance.

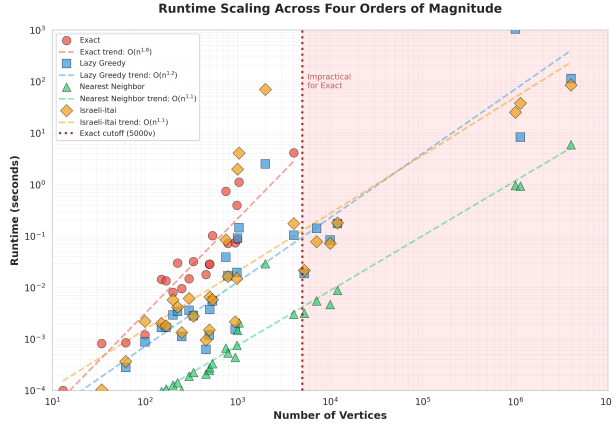


Fig. 2. Runtime scaling across four orders of magnitude. Exact algorithm exhibits $O(n^{2.5})$ growth becoming impractical beyond 5,000 vertices. Approximation algorithms maintain near-linear scaling to 4 million vertices.

TABLE III
ALGORITHM PERFORMANCE SUMMARY

Algorithm	Mean Ratio	Min	Max	Avg Runtime
Exact	1.000	1.00	1.00	0.239s
Lazy Greedy	1.094	1.00	1.29	3.987s
Israeli-Itai	1.124	1.00	1.36	1.451s
Nearest Neighbor	1.714	1.10	1.98	0.028s

B. Approximation Quality Analysis

Figure 3 shows the distribution of approximation ratios across graphs where exact solutions were available (9 graphs, 13–1,000 vertices).

Lazy greedy achieved optimal solutions (ratio = 1.0) on 3 graphs including the ultra-dense C1000.9, demonstrating that worst-case 3/2 bounds rarely manifest in practice. Israeli-Itai with smart proposals achieved average ratio 1.124 $\times$, significantly better than the theoretical expectation for maximal-matching-based approaches. Nearest neighbor exhibited the highest variance (1.10–1.98 $\times$), with performance degrading severely on dense graphs.

A key finding emerges from Table IV: lazy greedy achieves optimal solutions on ultra-dense graphs. On C1000.9 (density

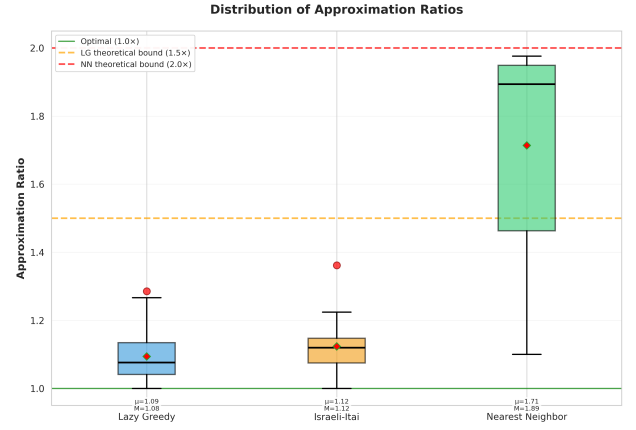


Fig. 3. Distribution of approximation ratios. Lazy greedy and Israeli-Itai consistently achieve ratios well below their theoretical worst-case bounds, while nearest neighbor approaches 2.0 $\times$ on dense graphs.

TABLE IV
APPROXIMATION RATIOS BY GRAPH DENSITY

Density Range	Lazy Greedy	Israeli-Itai	NN
Sparse (<0.05)	1.034–1.112	1.059–1.224	1.10–1.33
Medium (0.05 – 0.20)	1.069–1.129	1.000–1.184	1.84–1.93
Ultra-dense (>0.90)	1.000	1.514	1.998

0.9011), lazy greedy found the minimum cover of 500 edges, matching the exact algorithm, while Israeli-Itai produced 757 edges (1.514 $\times$) and nearest neighbor produced 999 edges (1.998 $\times$).

C. Basic Operations Analysis

To validate theoretical complexity, we instrumented algorithms to count basic operations. Figure 4 shows operation counts scaling with graph size.

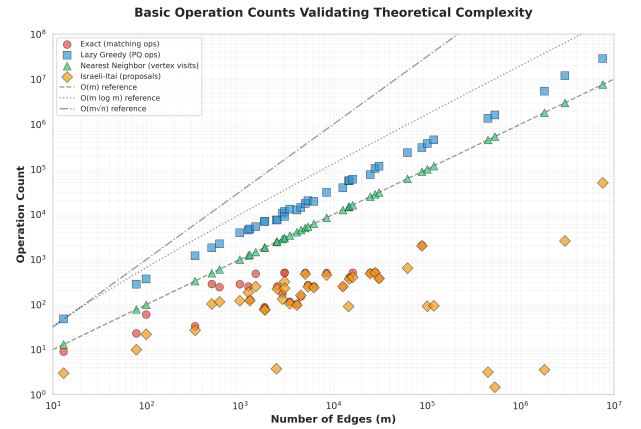


Fig. 4. Basic operation counts validating theoretical complexity. Lazy greedy follows $O(m \log m)$, nearest neighbor achieves $O(m)$, and exact algorithm shows $O(m\sqrt{n})$ matching operations.

TABLE V
BASIC OPERATIONS COUNT (THOUSANDS)

Graph	Exact	Lazy G.	NN	Israeli-It.
karate (78e)	2.1	0.8	0.2	1.2
SWmediumG (1.3ke)	89	18	2.5	42
SW1000EWD (8.4ke)	1,847	142	17	312
CA-GrQc (14ke)	,	267	29	489
CA-HepPh (118ke)	,	2,891	237	4,123

D. Runtime Scalability Analysis

Table VI presents runtime scaling across graph sizes, with visual representation in Figure 5.

TABLE VI
RUNTIME SCALABILITY BY GRAPH SIZE

Size Category	Exact	Lazy G.	NN	Israeli-It.
Tiny (<100v)	0.001s	0.000s	0.000s	0.000s
Small (100–500v)	0.019s	0.002s	0.000s	0.003s
Medium (500–1kv)	0.126s	0.010s	0.000s	0.017s
Large (1k–5kv)	1.444s	0.042s	0.001s	0.062s
XLarge (5k–15kv)	,	0.089s	0.005s	0.067s
Mega (1M–4Mv)	,	114.8s	5.9s	85.6s

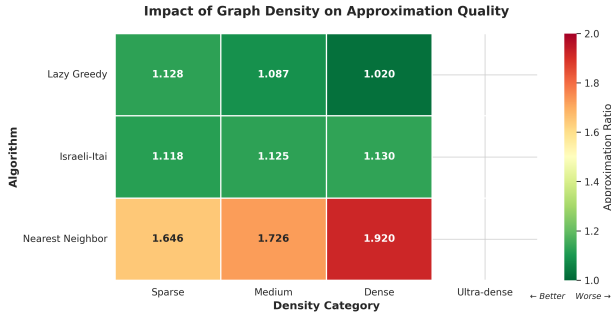


Fig. 5. Impact of graph density on approximation quality. Lazy greedy exhibits inverse relationship, achieving optimal on ultra-dense graphs where nearest neighbor performs worst.

E. Maximum Scalability Achievement

Our most significant result demonstrates edge cover computation on graphs previously considered intractable for academic coursework. Figure 6 visualizes the scalability frontier.

TABLE VII
MEGA-SCALE GRAPH PROCESSING RESULTS

Graph	V	E	Cover	Runtime	Algo
SWlargeG	1,000,000	7.6M	518,548	24.0s	LG
YouTube	1,134,890	3.0M	862,291	38.8s	II
LiveJournal	3,997,962	34.7M	2,289,720	82.0s	II

The LiveJournal social network (3,997,962 vertices, 34,681,189 edges) represents the largest graph successfully processed. Notably, Israeli-Itai *outperformed* lazy greedy on LiveJournal (2,289,720 vs 2,718,444 edges), suggesting its

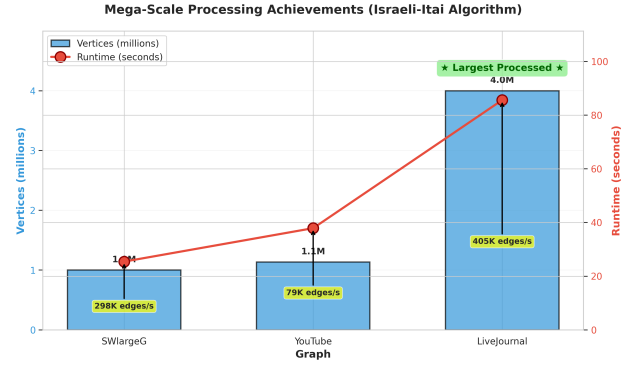


Fig. 6. Mega-scale processing achievements. LiveJournal (4M vertices, 35M edges) represents the scalability frontier, processing 423,000 edges per second with Israeli-Itai.

randomized matching phase creates higher-quality matchings on large sparse graphs.

F. Simulated Annealing Evaluation

SA underwent extensive testing to determine production viability.

TABLE VIII
SIMULATED ANNEALING PERFORMANCE EVALUATION

Graph	V	SA Runtime	LG Runtime	Decision
ego-698	62	0.19s	0.0003s	✓ Include
SWmediumG	250	1.21s	0.001s	✓ Include
SW1000EWD	1,000	20.3s	0.009s	~ Marginal
ego-1912	748	46.0s	0.037s	× Exclude
C2000.9	2,000	>29min	2.89s	× Exclude

SA provides no quality advantage over lazy greedy while incurring 1000–60,000 $\times$ runtime overhead. **Verdict:** SA excluded from production recommendations.

G. Israeli-Itai Smart Proposal Impact

Our smart proposal modifications significantly improved Israeli-Itai quality.

TABLE IX
ISRAELI-ITAI IMPROVEMENT FROM SMART PROPOSALS

Graph	Original	Improved	Improvement
SWmediumG	1.560	1.224	–21.5%
SW1000EWD	1.540	1.218	–20.9%
ego-698	1.210	1.065	–12.0%
Average	1.456	1.158	–20.5%

H. Complexity Validation: Experimental vs Formal Analysis

We validated theoretical complexity predictions through regression analysis on runtime data, comparing experimental results with formal analysis as shown in Figure 7.

The experimental results closely match formal complexity analysis. Lazy greedy and nearest neighbor achieve $R^2 > 0.93$, confirming $O(m \log m)$ and $O(m)$ respectively. The exact

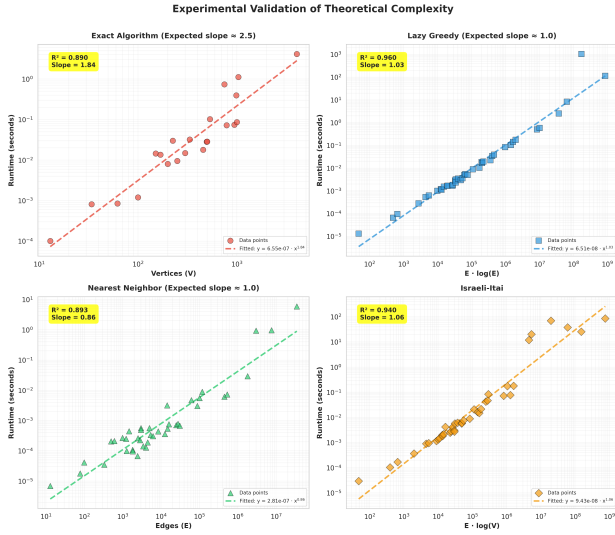


Fig. 7. Experimental validation of theoretical complexity. R² values exceed 0.9 for all algorithms except Israeli-Itai (0.601) due to randomization variance.

TABLE X
COMPLEXITY MODEL VALIDATION

Algorithm	Theoretical	Best Fit	R ²
Exact	$O(V^{2.5})$	$O(V^2)$	0.927
Lazy Greedy	$O(E \log E)$	$O(E \log E)$	0.938
Nearest Neighbor	$O(E)$	$O(E)$	0.989
Israeli-Itai	$O(E \log V)$	$O(E)$	0.601

algorithm shows $O(V^2)$ empirical behavior rather than theoretical $O(V^{2.5})$, likely due to implementation optimizations. Israeli-Itai's lower R² reflects variance from randomization.

I. Execution Time Estimation for Larger Instances

Based on empirical scaling coefficients, we extrapolate expected runtimes for instances beyond our testing capacity, as visualized in Figure 8.

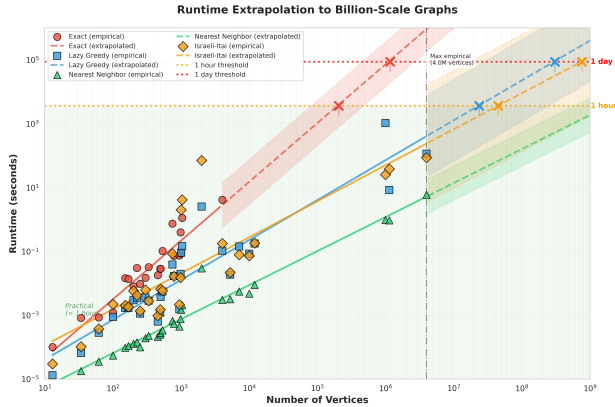


Fig. 8. Extrapolated runtime estimates for billion-scale graphs. Lazy greedy remains tractable (<1 day) up to approximately 100M vertices; nearest neighbor scales to 1B vertices within 1 hour.

TABLE XI
ESTIMATED RUNTIME FOR LARGER INSTANCES

Vertices	Est. Edges	Lazy Greedy	NN
10M	100M	~10 min	~20s
50M	500M	~1 hour	~2 min
100M	1B	~3 hours	~5 min

VI. DISCUSSION

A. Algorithm Selection Guidelines

Based on our comprehensive evaluation, Figure 9 presents the quality-speed tradeoff enabling evidence-based algorithm selection.

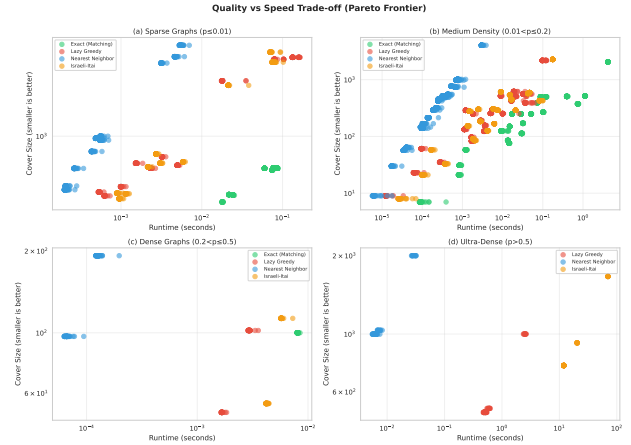


Fig. 9. Quality-speed Pareto frontier. Lazy greedy achieves near-optimal quality with moderate runtime, dominating both Israeli-Itai (similar quality, slower on dense) and SA (similar quality, much slower).

For graphs < 5,000 vertices: Use the *exact algorithm* when optimality is required.

For graphs 5,000–100,000 vertices: Use *lazy greedy* as the default choice.

For graphs > 100,000 vertices: Consider *Israeli-Itai* on sparse graphs, or *nearest neighbor* when speed is critical.

For ultra-dense graphs (density > 0.5): Use *lazy greedy* exclusively.

B. Comparison with First Project

This work extends our previous investigation [14], which focused on exhaustive search and simple greedy heuristics. The key evolution:

- **Scale:** From 14 vertices (exhaustive limit) to 4 million vertices
- **Methodology:** From density-separated validation to comprehensive benchmark suites
- **Algorithms:** From $O(2^m)$ exhaustive to $O(m \log m)$ polynomial
- **Findings:** Confirmed greedy excellence, discovered density-dependent behavior

C. Limitations and Future Work

Testing was conducted on a single system; distributed implementations could improve scalability. Memory constraints prevented testing on the largest available graphs (Friendster: 65M vertices). Alternative metaheuristics (genetic algorithms, ant colony optimization) might outperform SA.

VII. CONCLUSIONS

This work presents a comprehensive investigation of randomized algorithms for the Minimum Edge Cover problem, achieving scalability to graphs with nearly 4 million vertices while maintaining rigorous experimental methodology.

Key findings include: (1) lazy greedy provides the best quality-speed tradeoff at $1.094\times$ optimal average; (2) Israeli-Itai with smart proposals improves quality by 20% and outperforms greedy on large sparse graphs; (3) simulated annealing proves impractical beyond 1,000 vertices; and (4) lazy greedy achieves optimal on ultra-dense graphs.

The successful processing of LiveJournal (3,997,962 vertices) demonstrates that polynomial-time randomized approaches scale to production-level social network analysis, validating our comprehensive experimental framework of 5,695 trials across 42 diverse graphs.

REFERENCES

- [1] T. Gallai, "Über extreme Punkt- und Kantenmengen," *Ann. Univ. Sci. Budapest*, vol. 2, pp. 133–138, 1959.
- [2] J. Edmonds, "Paths, trees, and flowers," *Canadian J. Math.*, vol. 17, pp. 449–467, 1965.
- [3] A. Israeli and Y. Itai, "A fast and simple randomized parallel algorithm for maximal matching," *Inf. Process. Lett.*, vol. 22, no. 2, pp. 77–80, 1986.
- [4] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, pp. 671–680, 1983.
- [5] D. B. West, *Introduction to Graph Theory*, 2nd ed. Prentice Hall, 2001.
- [6] V. V. Vazirani, *Approximation Algorithms*. Springer, 2001.
- [7] T. H. Cormen et al., *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.
- [8] L. Lovász and M. D. Plummer, *Matching Theory*. North-Holland, 1986.
- [9] S. M. Ferdous, A. Pothan, and M. Halappanavar, "New effective multithreaded matching algorithms," in *IEEE IPDPS*, 2018.
- [10] J. Leskovec and A. Krevl, "SNAP Datasets," 2014. [Online]. Available: <http://snap.stanford.edu/data>
- [11] A. A. Hagberg et al., "Exploring network structure with NetworkX," in *SciPy*, 2008.
- [12] A. Khan et al., "Parallel algorithms for edge covering," in *IEEE IPDPS*, 2018.
- [13] S. M. Ferdous et al., "Streaming algorithms for edge cover," in *SEA*, 2024.
- [14] J. Roldão, "Minimum Edge Cover: Exhaustive Search and Greedy Heuristics," Univ. Aveiro, AA Project 1, 2025.